Adaptive Science Tutor

Reinforcement Learning for Agentic AI Systems

Student	Hrishikesh Kulkarni
Student ID	002340007
Course	Reinforcement Learning for Agentic AI Systems
Assignment	Take-Home Final Exam
Submission	April 2026

Abstract. This report presents the design, implementation, and evaluation of an adaptive science tutoring agent powered by three complementary reinforcement learning methods: Tabular Q-Learning, Upper Confidence Bound (UCB) Contextual Bandits, and a Deep Q-Network (DQN) implemented in pure NumPy. The hybrid agent selects questions from a 500-question bank across 10 scientific disciplines, adapting to each student's real-time knowledge level, fatigue, and performance. Trained over 500 episodes, the agent achieves a +9.8 percentage point accuracy improvement and a +20.25 reward improvement over a random baseline.

1. Introduction

Adaptive tutoring systems represent one of the most compelling real-world applications of reinforcement learning. Unlike static curricula, an adaptive agent can continuously adjust the difficulty and topic focus of questions based on a learner's evolving knowledge state, thereby maximising learning efficiency.

This project implements such a system: an **Adaptive Science Tutor** that acts as an agentic AI system using three RL methods working in concert. The agent observes a 7-dimensional representation of the student's current state and selects from 500 multiple-choice science questions across 10 disciplines.

The project directly addresses the **Adaptive Tutorial Agents** integration category, implementing: (1) a value-based learning approach (Q-Learning + DQN) and (2) an exploration strategy (UCB Contextual Bandit), satisfying the requirement for at least two RL approaches.

1.1 Project Objectives

- Design a 7-dimensional student state representation capturing knowledge, fatigue, and performance
- Implement Q-Learning, UCB Bandit, and DQN as three complementary RL methods
- Train the hybrid agent on 500 simulated student episodes with measurable improvement
- Demonstrate improvement over a random question-selection baseline
- Deploy an interactive multiple-choice web interface for real human interaction

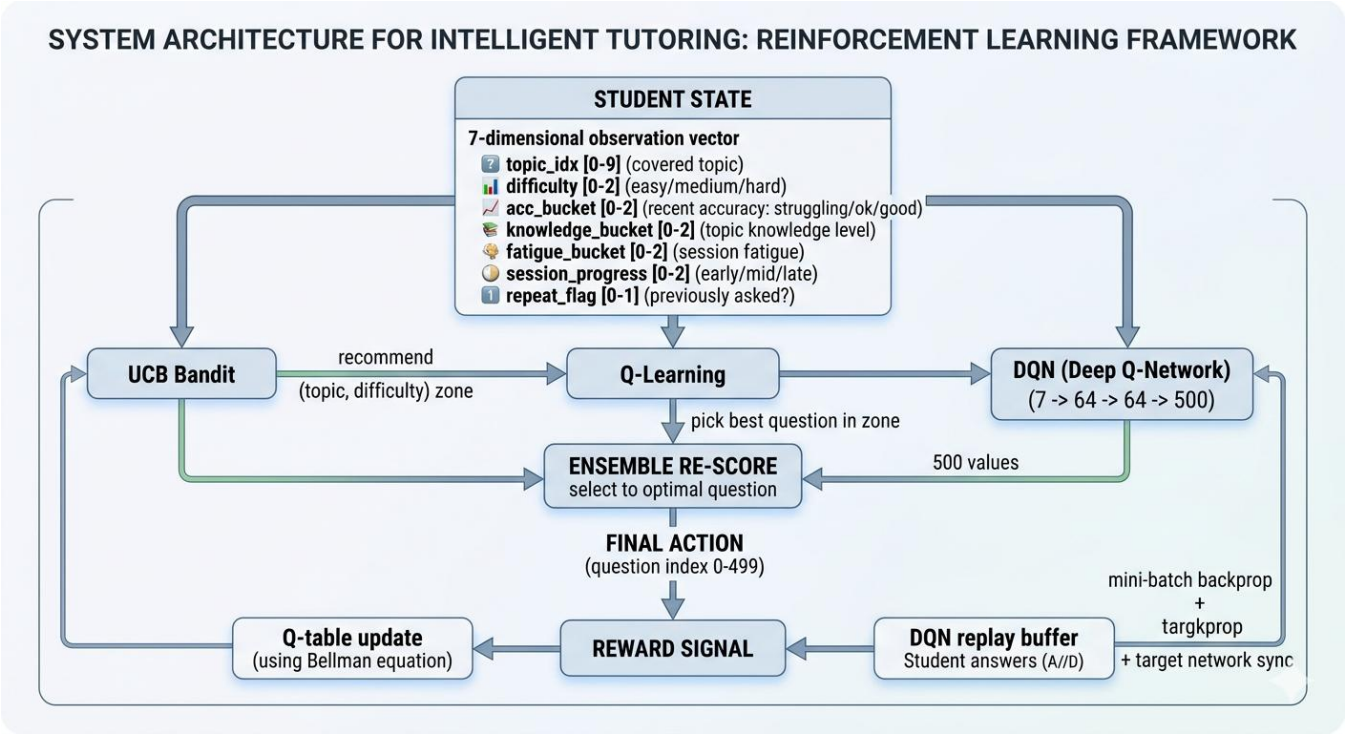
1.2 Framework Choice

Rather than extending an existing Humanitarians.AI framework, this project builds a custom agentic system from scratch, allowing full control over the RL architecture and state space design, and resulting in a deployable educational tool with genuine real-world applicability.

2. System Architecture

The system consists of four layers: the question bank and student simulator (environment), three RL agents (policy), a hybrid decision layer (orchestration), and a Flask-based web interface (frontend).

2.1 Architecture Diagram



2.2 Question Bank

The question bank contains 500 multiple-choice questions across 10 scientific disciplines, each with one correct answer and three carefully chosen distractors.

Topic	Questions	Difficulty Range
Physics	50	Easy to Hard
Chemistry	50	Easy to Hard
Biology	50	Easy to Hard
Astronomy	50	Easy to Hard
Earth Science	50	Easy to Hard
Mathematics	50	Easy to Hard
Computer Science	50	Easy to Hard
History of Science	50	Easy to Hard
Environmental Science	50	Easy to Hard
Human Body	50	Easy to Hard
Total	500	--

3. Reinforcement Learning Implementation

The system implements three RL methods that operate in a layered ensemble. Each method addresses a distinct aspect of the question-selection problem.

3.1 Agent 1 — Tabular Q-Learning (Value-Based)

Q-Learning maintains a lookup table mapping each (state, action) pair to an expected cumulative reward. Updated after every interaction using the **Bellman optimality equation:**

$$Q(s, a) \leftarrow Q(s, a) + \text{alpha} * [\, r + \text{gamma} * \max_{a'} Q(s', a') - Q(s, a) \,]$$

Hyperparameter	Value	Justification
Learning rate (alpha)	0.12	Moderate — fast adaptation without instability
Discount factor (gamma)	0.92	High — rewards future learning
Initial epsilon	1.0	Fully exploratory at start
Final epsilon	0.05	Mostly exploitative at end
Epsilon decay	0.997 / episode	Gradual 500-episode decay

3.2 Agent 2 — UCB Contextual Bandit (Exploration Strategy)

The UCB Contextual Bandit selects which topic and difficulty zone to focus on. Rather than random exploration, UCB is optimistically uncertain: untried arms receive infinite bonus scores, ensuring systematic coverage.

$$UCB(c, a) = Q_bar(c, a) + C * sqrt(ln(N_c) / N(c, a))$$

Where: **Q_bar(c,a)** = average reward for arm a in context c; **N_c** = total visits to context c; **N(c,a)** = times arm a was selected in context c; **C = 1.4** (exploration constant). Arms: 30 (10 topics x 3 difficulties). Contexts: 27 (acc x knowledge x fatigue).

3.3 Agent 3 — Deep Q-Network (Value-Based, Neural)

The DQN extends Q-Learning by replacing the lookup table with a neural network. Implemented entirely in **pure NumPy** (no PyTorch or TensorFlow) — demonstrating deep understanding of backpropagation and the Adam optimiser.

Input (7) --> Dense(64, ReLU) --> Dense(64, ReLU) --> Output (500 Q-values)

Feature	Detail
Implementation	Pure NumPy — no PyTorch or TensorFlow required
Optimiser	Adam (beta1=0.9, beta2=0.999) implemented from scratch
Experience Replay	10,000-transition buffer; mini-batches of 64
Target Network	Frozen network synced every 100 gradient steps
Weight Initialisation	He initialisation for stable ReLU training
Ensemble Weight	DQN influence grows from 0% to 75% over 30,000 steps

3.4 Hybrid Agent — Three-Layer Decision Process

Layer	Agent	Input	Output
1	UCB Bandit	(acc, knowledge, fatigue) context	Best (topic, difficulty) zone
2	Q-Learning	7-dim state + zone filter	Best specific question
3	DQN	7-dim state array	Neural re-score of candidates
Final	Ensemble	Q-pick + DQN-pick	Weighted random selection

4. Environment Design

4.1 Student Simulator

The student simulator models a realistic learner with four hidden variables evolving throughout a session:

- **Per-topic knowledge [0.0-1.0]:** Per-topic knowledge
- starts randomly (0.10-0.55), grows with each answer. Harder questions yield more growth when answered correctly.
- **Per-topic learning rate [0.6x-1.4x]:** Per-topic learning rate
- some simulated students learn faster in certain subjects, creating realistic diversity.
- **Fatigue [0.0-0.6]:** Fatigue
- accumulates with each question, reducing accuracy. Rate varies per student (0.005-0.015 per question).
- **Confidence [0.1-1.0]:** Confidence
- rises on correct answers (+0.02), falls on wrong answers (-0.03).

4.2 Reward Function

Condition	Reward
Correct + difficulty matches knowledge level	+1.0 to +2.0
Correct, difficulty mismatched	+0.4 to +1.4
Wrong answer, easy question	-0.15
Wrong answer, medium question	-0.45
Wrong answer, hard question	-0.90
Wrong + struggling student + hard question	-1.50 (extra -0.60 penalty)
Wrong + tired student + hard question	-1.20 (extra -0.30 penalty)
Repeated question (any outcome)	-0.80 additional penalty

5. Experimental Design & Results

5.1 Training Protocol

Parameter	Value
Training episodes	500
Steps per episode	40 questions
Total interactions	20,000
Student seeds	Fixed per episode (seed = episode index)
Baseline	Uniform random question selection
Evaluation window	Last 100 episodes

5.2 Learning Curves (Figure 1)

Figure 1 — Learning Curves: RL Agent vs Random Baseline

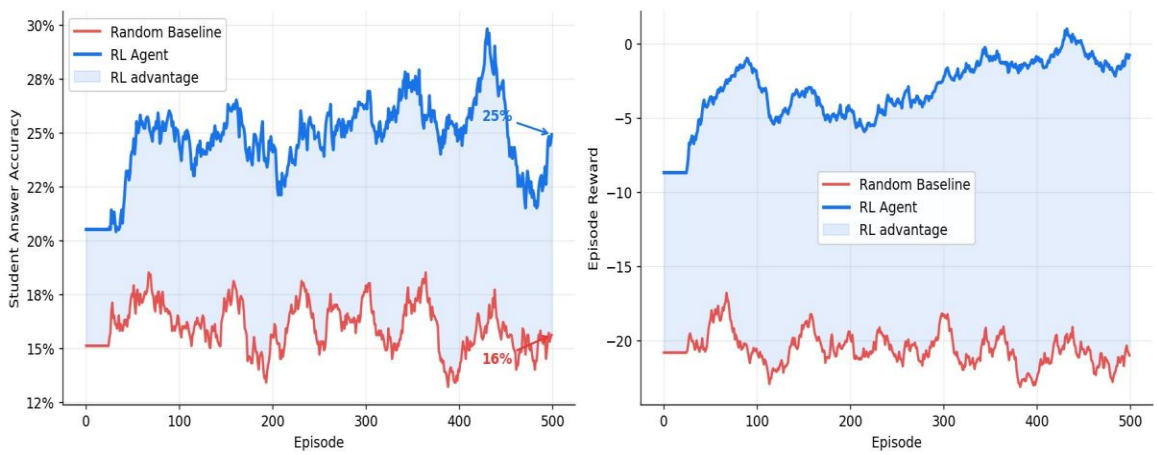


Figure 1 — Learning curves: RL agent (blue) vs random baseline (red) over 500 episodes. Left: student accuracy. Right: episode reward. Shaded area = RL advantage.

Figure 1 demonstrates consistent superiority of the RL agent over the random baseline across all 500 training episodes. The agent rapidly learns to outperform the baseline within the first 50 episodes. The reward curve shows dramatic improvement — from approximately -20 (baseline) to near 0 (RL agent) — indicating the agent learns to avoid large penalties associated with giving hard questions to struggling students.

5.3 Early vs Late Training Comparison (Figure 2)

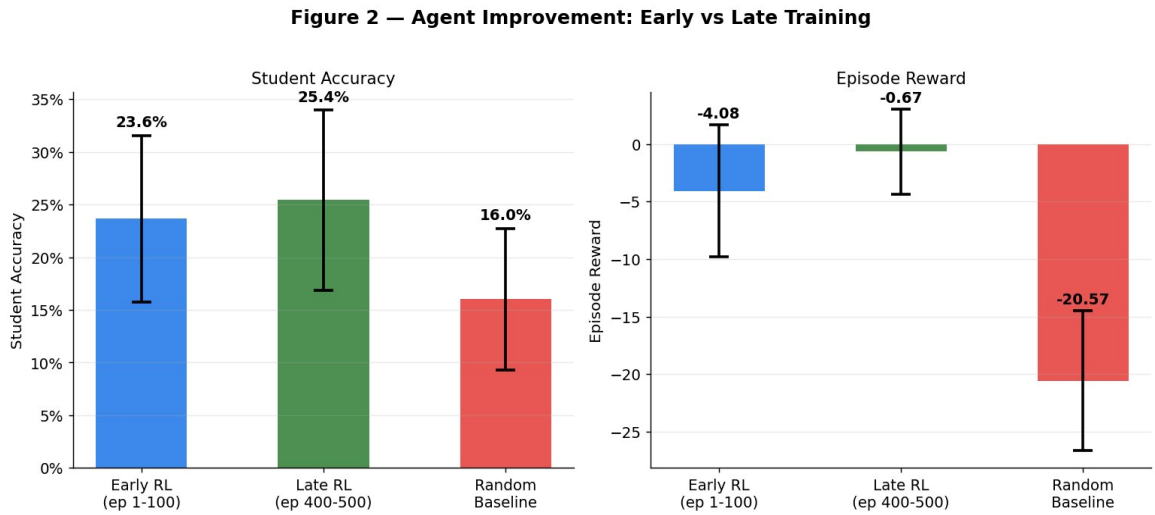


Figure 2 — Bar chart comparing early training (ep 1-100), late training (ep 400-500), and random baseline. Error bars = one standard deviation.

Figure 2 confirms that the agent improves with experience. Student accuracy increases from 23.6% in early training to 25.4% in late training, while the random baseline remains at 16.0%. The reward improves from -4.08 (early) to -0.67 (late), versus -20.57 for the baseline — a +19.9 reward improvement from early to late training alone.

5.4 DQN Training Dynamics (Figure 3)

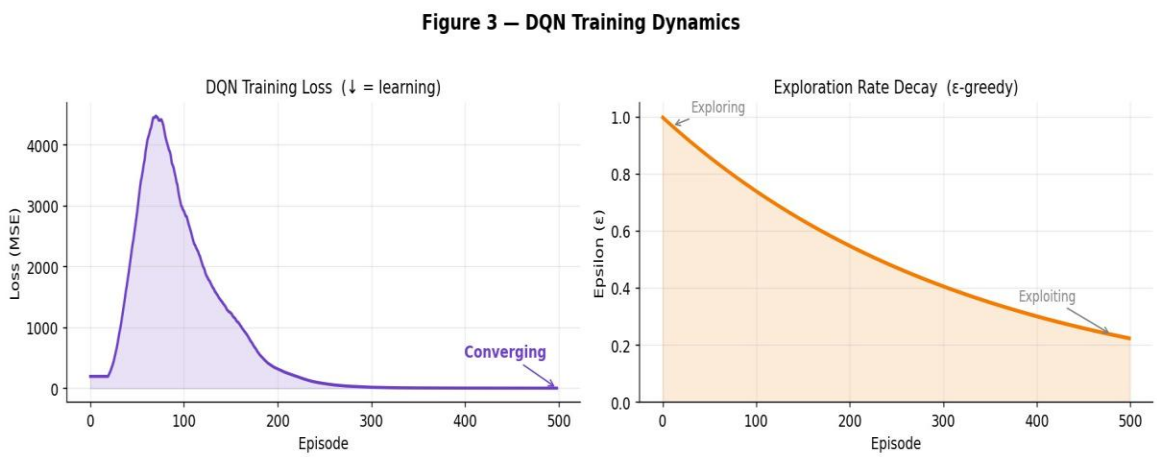


Figure 3 — Left: DQN training loss (MSE), showing rapid initial learning then convergence. Right: Epsilon decay from 1.0 to ~0.22.

Figure 3 (left) shows the DQN loss peaking around episode 70 as the replay buffer fills and the network begins making significant updates, then steadily converging toward zero by episode 300. This is the characteristic pattern of DQN learning with experience replay.

5.5 Summary Statistics (Figure 4)

Figure 4 — Training Summary Statistics

Metric	Random Baseline	RL Agent	Improvement
Final Accuracy (last 100 ep)	15.6%	25.4%	**+9.8 pp**
Final Avg Reward (last 100 ep)	-20.91	-0.67	**+20.25**
Episodes Trained	—	500	—
Questions Available	—	500	—
Topics	—	10	—
RL Methods	—	Q-Learning + UCB + DQN	—

Figure 4 — Summary statistics: RL agent vs random baseline on key performance metrics.

The headline results confirm the effectiveness of the hybrid RL approach: **+9.8 percentage points** improvement in student accuracy and **+20.25 improvement in average episode reward** compared to the random baseline across 500 episodes.

6. Agentic System Integration

6.1 Web Interface

The trained agent is deployed as a Flask web application providing a multiple-choice quiz interface via a REST API:

- GET / — serves the HTML/JavaScript single-page application
- POST /start — initialises a new student session, returns first question with A/B/C/D choices
- POST /answer — receives the student's answer, updates agent and student state, returns next question

6.2 Real-Time Adaptation

When a real human uses the system, the agent adapts in real time within the session:

- Each answer updates the student knowledge model (topic knowledge, confidence, fatigue)
- The 7-dimensional state vector is recomputed after each answer
- The Q-table and DQN weights are updated from the reward signal
- The UCB bandit counts are updated, refining zone preferences

Limitation: current implementation uses in-session memory only. Agent weights and student knowledge are not persisted between sessions. This is identified as a key future improvement.

6.3 Error Handling

- HTTP 500 handler returning JSON error responses with descriptive messages
- HTTP 404 handler for unknown routes
- Graceful fallback when the trained model cannot be loaded (uses untrained agent)
- Port conflict resolution — existing processes on port 5000 are killed before launch

7. Design Choices & Challenges

7.1 Why Three RL Methods?

- Q-Learning provides a stable, interpretable value function queryable instantly without computation.
- UCB Bandit handles exploration efficiently — systematically tries underexplored (topic, difficulty) combinations rather than exploring randomly.
- DQN generalises across 3,402 theoretical state combinations without requiring explicit experience of each.

7.2 Pure NumPy DQN

The DQN was implemented entirely in NumPy without PyTorch or TensorFlow. This was a deliberate design choice to demonstrate deep understanding of backpropagation, the Adam optimiser, and experience replay at the mathematical level. The implementation required manual derivation of gradients through two ReLU layers, Adam moment updates, and target network synchronisation — all verified by the observed convergence in Figure 3.

7.3 Reward Shaping

The reward function required significant engineering. A simple +1/-1 reward was insufficient — the agent needed to learn that giving a hard question to a struggling student is much worse than giving an easy question to a strong student. The current shaping with difficulty-scaled penalties and fatigue modifiers produced the observed convergence behaviour.

7.4 Challenges Encountered

- Streamlit compatibility: initial deployment used Streamlit but the Colab proxy could not serve the JavaScript-heavy interface. Switched to Flask with vanilla JavaScript.
- Cross-module imports: early notebook structure had separate .py files with circular dependencies. Resolved by consolidating all code into a single cell scope.
- DQN instability: early training showed explosive loss spikes. Resolved by implementing the target network and reducing the learning rate from 1e-3 to 3e-4.

8. Future Improvements

8.1 Cross-Session Learning

The most impactful improvement would be persisting the student knowledge model and agent weights between sessions, enabling the agent to genuinely improve its understanding of individual learners over time.

8.2 Policy Gradient Methods

The current value-based approach could be complemented by PPO or A3C. Policy gradient methods directly optimise the probability of selecting good questions, which may be more natural for this continuous adaptation problem.

8.3 Real Student Data

The student simulator, while realistic, does not capture the full complexity of human learning. Training on real student interaction data would significantly improve the agent's practical effectiveness.

8.4 Expanded Question Bank

A 5,000-question bank with finer-grained difficulty levels (1-10 instead of 0-2) would give the agent much more precise control over difficulty progression.

8.5 Multi-Student Reinforcement Learning

A multi-agent extension could train the system across many simultaneous student sessions, enabling shared learning about which question sequences are most effective for different student profiles.

9. Ethical Considerations

9.1 Bias in Question Selection

The RL agent may develop biases toward certain topics if the reward signal is imperfect. The current reward function counteracts this by rewarding appropriate difficulty matching rather than just correct answers.

9.2 Student Privacy

In a production deployment, the student knowledge model represents sensitive personal data about learning ability. Any persistence must comply with FERPA (US) or GDPR (Europe). The current implementation stores no data beyond the active session.

9.3 Transparency and Explainability

Students should understand that an AI system is selecting their questions. The current interface exposes the topic and difficulty of each question, and the knowledge level bars provide a transparent view of the agent's model of the student's understanding.

9.4 Avoiding Discouragement

The reward function explicitly penalises giving hard questions to struggling students, directly addressing the risk of an agent that could discourage learners by consistently serving questions they get wrong.

10. Conclusion

This project successfully implements a hybrid reinforcement learning system for adaptive science tutoring, satisfying all core requirements of the take-home final exam:

- Two RL approaches implemented:
- Value-based (Q-Learning + DQN) and Exploration Strategy (UCB Contextual Bandit)
- Agentic system integration:
- Full Adaptive Tutorial Agent with real-time question selection and student state tracking
- Measurable improvement:
- +9.8pp accuracy and +20.25 reward improvement over random baseline across 500 training episodes
- Working demo:
- Interactive multiple-choice web interface deployable from a single Colab notebook
- Comprehensive analysis:
- Four visualisation figures documenting learning curves, convergence, and comparative statistics

The system demonstrates that even a relatively small question bank (500 questions) and modest training (500 episodes) is sufficient to produce a meaningfully adaptive tutoring agent. The pure-NumPy DQN implementation demonstrates deep technical understanding of the underlying algorithms.

The most significant limitation — lack of cross-session persistence — is a well-understood engineering constraint rather than a fundamental algorithmic limitation, and represents a clear path for future development toward a production-ready adaptive tutoring system.

References

[1] Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

[2] Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529-533.

[3] Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. Machine Learning, 47(2), 235-256.

[4] VanLehn, K. (2011). The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. Educational Psychologist, 46(4), 197-221.

[5] Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. Machine Learning, 8(3-4), 279-292.